

Guia de Deploy Docker na VPS

Este guia mostra como subir o Sistema Clinico MVP em uma VPS usando Docker Compose.

Arquitetura

O deploy usa quatro containers:

- `frontend`: aplicacao Nuxt exposta para acesso publico.
- `backend`: API Flask rodando com Gunicorn, acessivel apenas pela rede interna Docker.
- `mysql`: banco MySQL persistido em volume Docker.
- `redis`: cache Redis persistido em volume Docker.

O Firebird/SPDATA fica fora do Docker e deve estar acessivel pela VPS via rede.

Arquivos adicionados

- `docker-compose.yml`: orquestra todos os servicos.
- `.env.example`: modelo das variaveis de producao.
- `backend/Dockerfile`: imagem do Flask/Gunicorn.
- `backend/.dockerignore`: evita copiar arquivos locais para a imagem.
- `frontend/Dockerfile`: imagem do Nuxt em producao.
- `frontend/.dockerignore`: evita copiar `node\_modules`, builds locais e `.env`.

Antes de publicar

1. Remova credenciais reais do repositorio.
2. Nao versiona arquivos `.env`.
3. Troque senhas que ja tenham sido compartilhadas ou commitadas.
4. Confirme que a VPS consegue acessar o servidor Firebird/SPDATA.
5. Confirme que a porta `80` da VPS esta liberada.

Importante: existe um arquivo local chamado `VPS - Acesso.md` com credenciais. Remova esse arquivo do repositorio e troque a senha da VPS antes de usar em producao.

Instalar Docker na VPS

Os comandos abaixo consideram uma VPS Ubuntu/Debian com acesso root.

```
```bash
apt update
apt install -y ca-certificates curl gnupg
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" >
/etc/apt/sources.list.d/docker.list
apt update
apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
docker --version
docker compose version
```
```

Se a VPS for Debian e o repositorio acima falhar, use o instalador oficial simplificado:

```
```bash
```

```
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```
```

Enviar o projeto para a VPS

Use Git ou copie o projeto para a VPS. Exemplo usando Git:

```
```bash
cd /opt
git clone URL_DO_SEU_REPOSITORIO sistema-clinico-mvp
cd sistema-clinico-mvp
```
```

Se o projeto já existir na VPS:

```
```bash
cd /opt/sistema-clinico-mvp
git pull
```
```

Configurar variáveis de ambiente

Crie o arquivo `.env` na raiz do projeto a partir do exemplo:

```
```bash
cp .env.example .env
nano .env
```
```

Preencha as variáveis:

```
```env
FRONTEND_PORT=80
NUXT_ENABLE MOCK_AUTH=false
TZ=America/Sao_Paulo
GUNICORN_WORKERS=3
GUNICORN_TIMEOUT=120

MYSQL_DATABASE=sistema_clinico_mvp
MYSQL_USER=clinico
MYSQL_PASSWORD=senha_forte_do_mysql
MYSQL_ROOT_PASSWORD=senha_forte_do_root_mysql

SECRET_KEY=senha_forte_para_flask
JWT_SECRET_KEY=senha_forte_para_jwt

FIREBIRD_HOST=ip_ou_host_do_firebird
FIREBIRD_PORT=3050
FIREBIRD_DATABASE=/caminho/para/o/banco.fdb
FIREBIRD_USER=usuario_firebird
FIREBIRD_PASSWORD=senha_firebird
FIREBIRD_CHARSET=WIN1252
```
```

Observações:

- ``MYSQL_DATABASE``, ``MYSQL_USER`` e ``MYSQL_PASSWORD`` são usados pelo Flask via ``SQLALCHEMY_DATABASE_URI``.
- ``NUXT_FLASK_BASE_URL`` já é definido no ``docker-compose.yml`` como ``http://backend:5000``.
- ``NUXT_ENABLE MOCK_AUTH=false`` impede login mockado em produção.
- ``TZ=America/Sao_Paulo`` mantém backend, frontend e MySQL no fuso esperado.

- `GUNICORN\_WORKERS` e `GUNICORN\_TIMEOUT` controlam o Gunicorn do backend sem rebuild da imagem.
- `FIREBIRD\_HOST` nao pode ser `localhost`, porque dentro do container `localhost` aponta para o proprio container.

Subir a aplicacao

Na raiz do projeto, execute:

```
```bash
docker compose up -d --build
```
```

Confira se os containers subiram:

```
```bash
docker compose ps
```
```

Os servicos `mysql`, `redis`, `backend` e `frontend` devem aparecer como `healthy` apos alguns segundos.

Rode as migrations do Flask no MySQL:

```
```bash
docker compose exec backend flask db upgrade
```
```

Veja os logs se precisar diagnosticar:

```
```bash
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f mysql
```
```

Depois disso, acesse:

```
```text
http://IP_DA_VPS
```
```

Liberar firewall

Se estiver usando `ufw`:

```
```bash
ufw allow OpenSSH
ufw allow 80/tcp
ufw enable
ufw status
```
```

O backend, MySQL e Redis nao precisam ser expostos publicamente.

Atualizar o deploy

Quando houver novas alteracoes no repositario:

```
```bash
cd /opt/sistema-clinico-mvp
git pull
docker compose up -d --build
docker compose exec backend flask db upgrade
```
```

```
docker image prune -f
```
```

## Parar e reiniciar

Parar todos os containers:

```
```bash
docker compose down
```
```

Reiniciar:

```
```bash
docker compose up -d
```
```

Reiniciar apenas um servico:

```
```bash
docker compose restart backend
docker compose restart frontend
```
```

## Backup do MySQL

Gerar backup:

```
```bash
docker compose exec mysql sh -c 'mysqldump -uroot -p"$MYSQL_ROOT_PASSWORD"
"$MYSQL_DATABASE"' > backup_mysql.sql
```
```

Restaurar backup:

```
```bash
docker compose exec -T mysql sh -c 'mysql -uroot -p"$MYSQL_ROOT_PASSWORD"
"$MYSQL_DATABASE"' < backup_mysql.sql
```
```

## Diagnostico rapido

Ver status:

```
```bash
docker compose ps
```
```

Ver logs do backend:

```
```bash
docker compose logs --tail=200 backend
```
```

Entrar no container do backend:

```
```bash
docker compose exec backend sh
```
```

Testar conexao Flask internamente:

```
```bash
docker compose exec frontend wget -qO- http://backend:5000/
```
```

...

Testar MySQL internamente:

```
```bash
docker compose exec mysql sh -c 'mysqladmin ping -h 127.0.0.1 -uroot
-p"$MYSQL_ROOT_PASSWORD"'
```
```

## Observacoes sobre dominio e HTTPS

Este Compose expoe o frontend em HTTP na porta `80`.

Para dominio com HTTPS, o ideal e adicionar um proxy reverso como Caddy ou Nginx na frente do frontend. Nesse caso, o Caddy/Nginx escutaria as portas `80` e `443`, e encaminharia para o container `frontend:3000`.

## Checklist final

- `.env` criado na VPS com senhas fortes.
- Senha da VPS rotacionada caso tenha sido compartilhada.
- `VPS - Acesso.md` removido do repositório.
- Firebird/SPDATA acessível a partir da VPS.
- Porta `80` liberada.
- `docker compose up -d --build` executado com sucesso.
- `docker compose exec backend flask db upgrade` executado com sucesso.
- Aplicação acessível pelo navegador.